PORTFOLIO ROADMAP

From Zero to Automotive AI Engineer

Traffic Sign Recognition - Project Blueprint

Prepared for your journey into Germany's automotive AI industry

July 2026

1. YOUR BACKGROUND (As You Told Me)

1.1 Personal Context

You graduated in Computer Engineering in Cameroon feeling underprepared - that classic impostor syndrome that hits most engineering graduates. You're now in Germany pursuing a Master's in Data Science, AI, and Digital Business. You want to build a portfolio that actually lands you a job at a German automotive company (BMW, Mercedes-Benz, Volkswagen, Audi, Bosch, ZF, Continental).

1.2 Current Skill Level

- Python: Can write basic scripts (functions, loops, using libraries with Google)
- ML/AI: Minimal practical experience, learning from Master's coursework
- Tools: No established workflow yet (no Docker, cloud, or deployment experience)
- Time: 10-20 hours/week available

1.3 Target Industry

German automotive companies are the backbone of the country's economy. They hire heavily for:

- Computer Vision engineers (perception systems, object detection, traffic sign recognition)
- Machine Learning engineers (predictive maintenance, quality control, demand forecasting)
- MLOps engineers (model deployment, monitoring, CI/CD pipelines)
- Data scientists (analytics, A/B testing, recommendation systems)

The first project you'll build - Traffic Sign Recognition - directly targets the Computer Vision / perception engineering path, which is the most in-demand role in automotive AI right now.

1.4 Core Philosophy of This Plan

We don't waste time on theory-for-theory's-sake. Every line of code you write should go into your portfolio. You learn by building real things. When you hit a gap in knowledge, you fill it just enough to unblock yourself. This is 'just-in-time learning' - the opposite of university's 'just-in-case learning'.

2. THE 12-WEEK ROADMAP

2.1 Overview

The plan is structured in three phases, each building on the last. By week 12, you'll have two production-quality portfolio projects and the confidence to talk about them in interviews.

PHASE 1: Foundation + First Project (Weeks 1-3)

Week 1: Environment Setup & Python Deep Dive

- Install Python, VS Code, Git, and set up a proper development environment
- Learn Git basics (clone, add, commit, push) - crucial for portfolio credibility
- Reinforce Python fundamentals: functions, classes, list comprehensions, file I/O
- Introduction to NumPy and Pandas for data manipulation
- DELIVERABLE: A clean GitHub account with your first repository

Week 2: Machine Learning Crash Course

- Core concepts: features, labels, training, testing, overfitting
- Build your first model with sklearn on a simple dataset (iris, breast cancer)
- Learn about train/test split, accuracy, confusion matrix
- Introduction to neural networks: what are they, how do they learn?
- DELIVERABLE: A simple classifier notebook on GitHub

Week 3: Deep Learning & CNNs

- What are Convolutional Neural Networks? (filters, pooling, fully connected layers)
- Introduction to PyTorch (preferred for CV) or TensorFlow/Keras
- Build a tiny CNN from scratch on MNIST (handwritten digits) - the 'Hello World' of CV
- Understand transfer learning: why you don't need to train from scratch
- DELIVERABLE: MNIST classifier + transfer learning notebook

PHASE 2: Traffic Sign Recognition (Weeks 4-8)

Week 4: Dataset Deep Dive

- Download and explore the GTSRB dataset (German Traffic Sign Recognition Benchmark)
- Explore classes, class imbalance, image sizes, visualization
- Data preprocessing: resizing, normalization, data augmentation
- Split into train/validation/test sets
- DELIVERABLE: Data exploration notebook with visualizations

Week 5: Model Building & Training

- Set up a PyTorch data pipeline (Dataset, DataLoader)
- Load a pretrained model (ResNet18/34 or EfficientNet)
- Fine-tune on GTSRB data (freeze base layers, train new classifier head)
- Training loop, loss tracking, learning rate scheduling
- DELIVERABLE: Training script with saved model checkpoint

Week 6: Evaluation & Optimization

- Evaluate on test set: accuracy per class, confusion matrix, precision/recall
- Error analysis: what signs does the model get wrong?
- Hyperparameter tuning (learning rate, batch size, optimizer choice)
- Experiment tracking (optional: log with TensorBoard or Weights & Biases)

- DELIVERABLE: Evaluation report + optimizations

Week 7: Real-Time Inference (Webcam Demo)

- Set up OpenCV to capture webcam feed
- Integrate the trained model for real-time classification
- Overlay predictions on the video feed with bounding boxes
- Handle frame rate optimization (model doesn't need to run every frame)
- DELIVERABLE: Working webcam demo script

Week 8: Polish & Document

- Clean up all code with proper structure (src/, notebooks/, models/, README.md)
- Write a compelling README.md with screenshots, setup instructions, results
- Create a requirements.txt for easy environment reproduction
- Optional: Wrap as a Streamlit web app for browser-based demo
- DELIVERABLE: Complete, polished GitHub repository

PHASE 3: Vehicle Damage Detection (Weeks 9-12)

Week 9: Object Detection Basics

- Understand object detection vs image classification
- Introduce YOLO (You Only Look Once) - state-of-the-art real-time detection
- Explore the Car Damage Detection dataset (available on Kaggle)
- Set up a YOLO detection pipeline

Week 10: Training & Fine-Tuning Damage Detector

- Annotate/format data for YOLO training
- Fine-tune a pretrained YOLO model on vehicle damage
- Evaluate mAP (mean Average Precision), IoU metrics

Week 11: Web App Backend (FastAPI)

- Create a FastAPI server that accepts image uploads
- Run model inference server-side
- Return structured JSON with damage type, location, severity
- Add basic frontend (HTML/Streamlit) for upload + visualization

Week 12: Deployment & Portfolio Polish

- Dockerize the application
- Deploy on a free cloud service (Render, Hugging Face Spaces, or Railway)
- Combine both projects into a portfolio website or single GitHub profile
- Write LinkedIn posts about each project
- DELIVERABLE: Two live, deployed projects + GitHub profile ready for applications

3. PROJECT 1: TRAFFIC SIGN RECOGNITION - DEEP DIVE

3.1 What Are We Building?

A real-time traffic sign classifier that takes video from your webcam and identifies German traffic signs as they appear. It uses a deep neural network fine-tuned on the German Traffic Sign Recognition Benchmark (GTSRB) dataset, which contains over 50,000 images of 43 different traffic sign classes.

3.2 Why This Project?

- Directly relevant to autonomous driving perception systems at BMW, Mercedes, VW
- Shows understanding of CNNs, transfer learning, real-time inference
- Visually impressive demo - easy to show in interviews
- Teaches skills that transfer to object detection, semantic segmentation, tracking
- GTSRB is a well-known benchmark; recruiters will recognize it

3.3 Technologies You'll Use

- Python 3.10+ - main programming language
- PyTorch - deep learning framework (preferred for CV research)
- OpenCV - camera capture and image processing
- TorchVision - pretrained models and image transforms
- NumPy / Matplotlib / Seaborn - data manipulation and visualization
- Jupyter Notebooks - exploration and prototyping
- Git / GitHub - version control and portfolio hosting

3.4 The GTSRB Dataset

The German Traffic Sign Recognition Benchmark (GTSRB) is a dataset collected by the Institut f. Neuroinformatik, Ruhr-Universitat Bochum. It contains:

- 50,000+ images of German traffic signs
- 43 classes (speed limits, stop, yield, no entry, etc.)
- Images vary in size (15x15 to 250x250 pixels)
- Real-world conditions: varying lighting, occlusion, rotation
- Official train/test split provided

The dataset is challenging because of class imbalance (some signs have few examples) and natural variation in the images. This makes it a realistic benchmark.

3.5 Key Concepts You'll Learn

Convolutional Neural Networks (CNNs)

CNNs are the backbone of modern computer vision. They work by applying filters (kernels) that slide over the image to detect features like edges, textures, and shapes. Early layers detect simple patterns, deeper layers detect complex structures (wheels, letters, sign shapes).

Transfer Learning

Instead of training a network from scratch (which requires millions of images and weeks of compute), we take a network already trained on ImageNet (1.4 million images) and 'fine-tune' it on our traffic signs. We freeze the early layers (general features) and retrain only the later layers (sign-specific features). This works well even with limited data.

Data Augmentation

To make our model robust, we artificially expand the dataset by applying random transformations: rotation, brightness changes, perspective warping, cropping. This teaches the model to be invariant to real-world conditions.

Real-Time Inference

Running a model at camera frame rate requires optimization. We learn about batch processing, frame skipping (run model every N frames), and model quantization if needed. This is exactly what autonomous driving systems do.

3.6 Expected Results

With proper transfer learning, you can expect:

- Test accuracy: 95-98% (state-of-the-art is ~99.7%)
- Inference speed: 30+ FPS on a laptop with GPU, 10+ FPS on CPU
- Robust to: lighting changes, slight rotations, partial occlusion

Don't chase the perfect 99% - anything above 95% with a live demo is impressive for a portfolio project.

4. WEEK 1: ENVIRONMENT SETUP - STEP BY STEP

4.1 Install Python

We'll use Python 3.10 or 3.11 (stable and widely compatible). Check if you already have it by running in your terminal:

```
python --version
```

If not installed, download from python.org. Make sure to check 'Add Python to PATH' during installation.

4.2 Install VS Code

Download from code.visualstudio.com. Essential extensions to install:

- Python (by Microsoft) - linting, debugging, IntelliSense
- Jupyter - run notebooks inside VS Code
- GitLens - visualize git history
- GitHub Pull Requests - manage PRs

4.3 Install Git

Download from git-scm.com. Then configure:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

4.4 Create GitHub Account

Sign up at github.com. Your GitHub profile IS your resume for tech roles. Use a professional username (firstnamelastname or similar). Add a profile photo and short bio.

4.5 Set Up Project Structure

Create a folder for the project with this structure:

```
traffic-sign-recognition/
  notebooks/       # Jupyter notebooks for exploration
  src/             # Production Python scripts
  models/          # Saved model checkpoints
  data/            # Dataset (gitignored if large)
  outputs/         # Figures, logs, results
  README.md        # Project documentation
  requirements.txt # Dependencies
```

4.6 Initialize Git and First Commit

```
cd traffic-sign-recognition
git init
echo "data/
*.pyc
__pycache__/
*.pt
*.pth
.vscode/" > .gitignore
git add .
git commit -m "Initial project structure"
```

4.7 Create Requirements File

Create requirements.txt with:

```
torch>=2.0.0
torchvision>=0.15.0
opencv-python>=4.8.0
numpy>=1.24.0
matplotlib>=3.7.0
seaborn>=0.12.0
pandas>=2.0.0
scikit-learn>=1.2.0
jupyter>=1.0.0
tqdm>=4.65.0
```

4.8 First Python Script

Create a test script to verify everything works:

```
# test_setup.py
import torch
import torchvision
import cv2
import numpy as np
import matplotlib.pyplot as plt

print(f"PyTorch: {torch.__version__}")
print(f"TorchVision: {torchvision.__version__}")
print(f"OpenCV: {cv2.__version__}")
print(f"NumPy: {np.__version__}")
print("All imports successful!")

# Quick sanity check
x = torch.randn(3, 224, 224)
print(f"Random tensor shape: {x.shape} - OK")
print(f"CUDA available: {torch.cuda.is_available()}")
```

Run it:

```
python test_setup.py
```

5. IMMEDIATE NEXT STEPS

Right now, do this:

- 1. Install Python, VS Code, Git (follow Section 4)
- 2. Create your GitHub account
- 3. Create the project folder with the structure above
- 4. Run the test script and confirm everything works
- 5. Push your project to GitHub
- 6. Message me when you're done

Remember: Every expert was once a beginner who didn't give up. The gap between 'I don't know shit' and 'I built this' is just consistent effort, one day at a time. Let's go.

This document was generated for you on July 5, 2026.

Keep it as a reference throughout the journey.